

mrpSteering

1 Executive Summary

The intend of this module is to implement an MRP attitude steering law where the control output is a vector of commanded body rates. To use this module it is required to use a separate rate tracking servo control module, such as `rateServoFullNonlinear`, as well.

2 Message Connection Descriptions

The following table lists all the module input and output messages. The module msg connection is set by the user from python. The msg type contains a link to the message structure definition, while the description provides information on what this message is used for.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
guidInMsg	<code>AttGuidMsgPayload</code>	Attitude guidance input message
rateCmdOutMsg	<code>RateCmdMsgPayload</code>	Rate command output message

3 Module Parameters

The following table lists all the module parameters than can be set. The parameters are optional unless indicated (if not specified default is used).

Table 2: Module Parameters

Parameter Name	Default	Description	Bounds
K1	0	Proportional gain applied to MRP errors	Must not be negative (checked in setter)
K3	0	Cubic gain applied to MRP errors	Must not be negative (checked in setter)
omegaMax (required)	0	Maximum rate command of steering control	Must be greater than zero (checked in setter)
ignoreOuterLoopFeedforward	false	Indicates if feedforward term should be included	N/A

4 Module Assumptions and Limitations

This control assumes the spacecraft is rigid, and that a fast enough rate control sub-servo system is present.

5 Initialization

The module is configured by:

```

module = mrpSteering.MrpSteering()
module.modelTag = "mrpSteering"
module.K1 = K1
module.K3 = K3
module.omegaMax = omega_max

```

If the feed-forward term should be ignored:

```
module.ignoreOuterLoopFeedforward = True
```

6 Detailed Module Description

The following text describes the mathematics behind the `mrpSteering` module. Further information can also be found in the journal paper *Speed-Constrained Three-Axes Attitude Control Using Kinematic Steering*.

6.1 Steering Law Goals

The goal of MRP Steering is to drive a body frame $\mathcal{B} : \{\hat{\mathbf{b}}_1, \hat{\mathbf{b}}_2, \hat{\mathbf{b}}_3\}$ towards a time varying reference frame $\mathcal{R} : \{\hat{\mathbf{r}}_1, \hat{\mathbf{r}}_2, \hat{\mathbf{r}}_3\}$:

$$\sigma_{\mathcal{B}/\mathcal{R}} \rightarrow 0$$

The attitude error $\sigma_{\mathcal{B}/\mathcal{R}}$ is provided by an upstream module such as `attTrackingError`.

6.2 MRP Steering Law

To create a kinematic steering law, let $\mathcal{B}^*$ be the desired body orientation, and $\omega_{\mathcal{B}^*/\mathcal{R}}$ be the desired angular velocity vector of this body orientation relative to the reference frame $\mathcal{R}$. The steering law requires an algorithm for the desired body rates $\omega_{\mathcal{B}^*/\mathcal{R}}$ relative to the reference frame.

The desired body rate is computed by

$${}^B\omega_{\mathcal{B}^*/\mathcal{R}} = -\mathbf{f}(\sigma_{\mathcal{B}/\mathcal{R}})$$

where $\mathbf{f}(\sigma)$ is an even function

$$f(\sigma_i) = \arctan \left((K_1\sigma_i + K_3\sigma_i^3) \frac{\pi}{2\omega_{\max}} \right) \frac{2\omega_{\max}}{\pi}$$

and with

$$\sigma_{\mathcal{B}/\mathcal{R}} = (\sigma_1, \sigma_2, \sigma_3)^T$$

and

$$\mathbf{f}(\sigma_{\mathcal{B}/\mathcal{R}}) = \begin{bmatrix} f(\sigma_1) \\ f(\sigma_2) \\ f(\sigma_3) \end{bmatrix}$$

The required velocity servo loop design is aided by knowing the body-frame derivative of ${}^B\omega_{\mathcal{B}^*/\mathcal{R}}$ to implement a feed-forward component. Using the $\mathbf{f}()$ function definition, this requires the time derivatives of $f(\sigma_i)$.

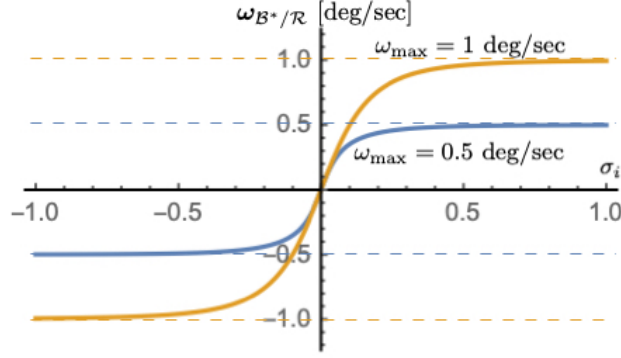


Figure 1: $\omega_{\max}$ dependency with $K_1 = 0.1$, $K_3 = 1$

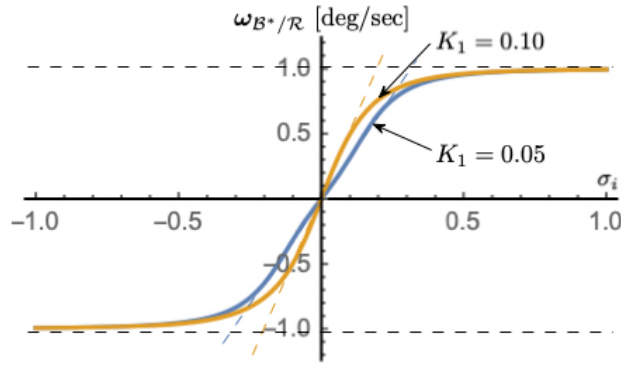


Figure 2: K_1 dependency with $\omega_{\max} = 1$ deg/s, $K_3 = 1$

$$\frac{{}^B d({}^B \omega_{B^*}/R)}{dt} = \omega'_{B^*}/R = -\frac{\partial f}{\partial \sigma_{B^*}/R} \dot{\sigma}_{B^*}/R = -\begin{bmatrix} \frac{\partial f}{\partial \sigma_1} \dot{\sigma}_1 \\ \frac{\partial f}{\partial \sigma_2} \dot{\sigma}_2 \\ \frac{\partial f}{\partial \sigma_3} \dot{\sigma}_3 \end{bmatrix}$$

where

$$\dot{\sigma}_{B^*}/R = \begin{bmatrix} \dot{\sigma}_1 \\ \dot{\sigma}_2 \\ \dot{\sigma}_3 \end{bmatrix} = \frac{1}{4} [B(\sigma_{B^*}/R)]^B \omega_{B^*}/R$$

Using the $f()$ definition, its sensitivity with respect to σ_i is

$$\frac{\partial f}{\partial \sigma_i} = \frac{(K_1 + 3K_3\sigma_i^2)}{1 + (K_1\sigma_i + K_3\sigma_i^3)^2 \left(\frac{\pi}{2\omega_{\max}}\right)^2}$$

If `ignoreOuterLoopFeedforward == true`, then $\omega'_{B^*}/R = 0$ and the computation of the feed-forward component is skipped.

Figures 1-3 illustrate how the parameters $\omega_{\max}$, K_1 and K_3 impact the steering law behavior. The maximum steering law rate commands are easily set through the $\omega_{\max}$ parameters. The gain K_1 controls the linear stiffness when the attitude errors have become small, while K_3 controls how rapidly the steering law approaches the speed command limit.

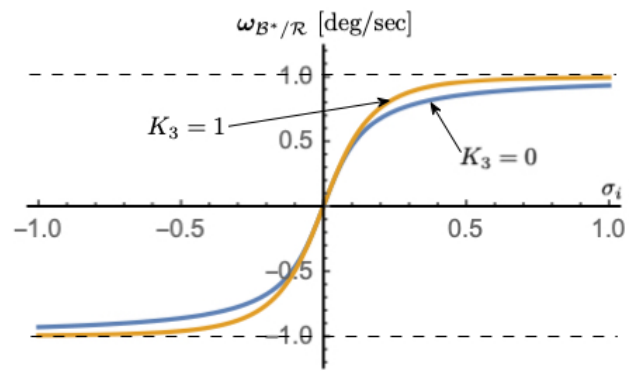


Figure 3: K_3 dependency with $\omega_{\max} = 1$ deg/s, $K_1 = 0.1$